

ui.json_editor 全面详解

`ui.json_editor` 是 NiceGUI 基于 JSONEditor 库封装的专业 JSON 编辑组件，核心用于网页端实现 JSON 数据的可视化编辑、校验、格式化与交互操作，支持 JSON Schema 校验、节点展开 / 折叠、只读模式切换等功能，适用于配置管理、数据编辑、接口调试等场景。以下从核心特性、配置参数、使用方法、进阶用法等维度展开全面解析。

一、核心定位与基础特性

1. 核心功能

- 可视化编辑：支持 JSON 数据的树形结构展示，可直接添加、删除、修改键值对，自动识别数据类型（字符串、数字、布尔值、数组、对象、null）。
- JSON Schema 校验：通过自定义 Schema 约束 JSON 数据结构（如字段类型、必填项、数值范围），编辑时实时提示非法数据。
- 交互操作：支持节点全展开 / 全折叠、数据导出、只读模式切换，选中内容时触发回调。
- 动态更新：可通过代码修改编辑器内容，或调用 JSONEditor 原生方法扩展功能。
- 样式定制：支持通过 Tailwind CSS 类修改编辑器容器样式，适配不同页面布局。

2. 基础用法

通过 `ui.json_editor()` 传入初始 JSON 数据（封装在 `properties['content']['json']` 中），快速创建 JSON 编辑器，示例代码结构如下：

```
from nicegui import ui

# 初始 JSON 数据
initial_json = {
    'array': [1, 2, 3],
    'boolean': True,
    'string': 'Hello NiceGUI',
    'object': {'a': 'b', 'c': 'd'},
    'number': 123,
    'null_value': None
}

# 创建 JSON 编辑器，绑定选择和变更事件
editor = ui.json_editor(
    properties={'content': {'json': initial_json}},
    on_select=lambda e: ui.notify(f'选中内容: {e}'),
    on_change=lambda e: ui.notify(f'数据已更新: {e}')
).classes('w-full h-96')

ui.run()
```

二、关键配置参数

`ui.json_editor` 的初始化参数用于定义编辑器的初始数据、校验规则、事件回调等，以下是完整参数说明：

参数名	类型	说明	默认值	版本特性
properties	字典	JSONEditor 核心配置，必填项，需包含 <code>content</code> 字段（指定初始 JSON 数据）	-	-
on_select	回调函数	选中 JSON 节点时触发，参数为 <code>JsonEditorSelectEventArguments</code>	-	-
on_change	回调函数	JSON 数据变更时触发，参数为 <code>JsonEditorChangeEventArguments</code>	-	-
schema	字典	JSON Schema 校验规则，用于约束数据结构	None	2.8.0 版本新增

核心参数补充说明

1. properties 配置详情

`properties` 字典直接传递给 JSONEditor 实例，支持其原生配置项，核心字段如下：

- `content.json`：初始 JSON 数据（必填）。
- 可选原生配置（参考 [JSONEditor 官方文档](#)）：
 - `mode`：编辑模式（默认 `'tree'`，支持 `'code'` 纯文本模式、`'form'` 表单模式等）。
 - `readOnly`：是否只读（默认 `False`）。
 - `indentation`：缩进空格数（默认 2）。

示例：配置纯文本编辑模式 + 4 空格缩进

```
editor = ui.json_editor(  
    properties={  
        'content': {'json': initial_json},  
        'mode': 'code',  
        'indentation': 4  
    }  
)  
.classes('w-full h-96')
```

2. JSON Schema 校验规则

`schema` 参数遵循 JSON Schema 规范，用于定义数据约束，常用关键字：

- `type`：字段类型（`object/array/string/number/boolean/null`）。
- `properties`：对象类型字段的子字段定义。
- `required`：必填字段列表。
- `exclusiveMinimum/exclusiveMaximum`：数值类型的范围约束（不含边界值）。
- `minLength/maxLength`：字符串类型的长度约束。

三、核心功能用法

1. 基础数据编辑与动态更新

(1) 修改编辑器内容

通过直接修改 `editor.properties['content']['json']` 动态更新 JSON 数据，示例：

```
from nicegui import ui
import random

# 初始数据
editor = ui.json_editor(
    properties={'content': {'json': {'number': 0}}},
    on_change=lambda e: ui.notify(f'数值更新为: {e.value["number"]}'),
).classes('w-full h-64')

# 按钮随机修改数值
ui.button('随机更新数值', on_click=lambda: editor.properties['content']['json'].update(
    number=random.randint(0, 100)
))

ui.run()
```

(2) 获取当前编辑数据

通过 `run_editor_method('get')` 异步获取编辑器当前的 JSON 数据，示例：

```
from nicegui import ui

initial_json = {'name': 'Alice', 'age': 42}
editor = ui.json_editor(properties={'content': {'json': initial_json}}).classes('w-full h-64')

# 异步获取数据并提示
async def get_current_data():
    data = await editor.run_editor_method('get')
    ui.notify(f'当前数据: {data}')

ui.button('获取数据', on_click=get_current_data)

ui.run()
```

2. JSON Schema 校验

通过 `schema` 参数设置校验规则，编辑时实时提示非法数据（如类型错误、必填项缺失、数值超出范围），示例：

```
from nicegui import ui

# 定义 Schema 规则：商品数据约束
product_schema = {
```

```

    'type': 'object',
    'properties': {
        'id': {'type': 'integer', 'description': '商品ID（必须为整数）'},
        'name': {'type': 'string', 'minLength': 2, 'description': '商品名称（至少2个字符）'},
        'price': {'type': 'number', 'exclusiveMinimum': 0, 'description': '价格（必须大于
0）'},
        'in_stock': {'type': 'boolean', 'description': '是否有货'}
    },
    'required': ['id', 'name', 'price'], # 必填字段
    'additionalProperties': False # 不允许额外字段
}

# 初始商品数据
initial_product = {'id': 42, 'name': 'Banana', 'price': 15.0, 'in_stock': True}

# 创建带校验的编辑器
editor = ui.json_editor(
    properties={'content': {'json': initial_product}},
    schema=product_schema
).classes('w-full h-96')

ui.run()

```

- 若将 `id` 改为字符串（如 `'42'`），编辑器会提示类型错误。
- 若删除 `name` 字段，会提示“必填项缺失”。
- 若将 `price` 设为 `0`，会提示“需大于 0”。

3. 节点展开 / 折叠与只读模式

通过 `run_editor_method` 调用 JSONEditor 原生方法，实现全展开、全折叠、切换只读模式，示例：

```

from nicegui import ui

initial_json = {
    'Name': 'Alice',
    'Age': 42,
    'Address': {
        'Street': 'Main Street',
        'City': 'Wonderland',
        'Contacts': {'Phone': '123456', 'Email': 'alice@example.com'}
    }
}

editor = ui.json_editor(properties={'content': {'json': initial_json}}).classes('w-full h-96')

# 功能按钮组
with ui.row():
    # 全展开（方法名前加 ":" 表示参数为 JavaScript 表达式）
    ui.button('全展开', on_click=lambda: editor.run_editor_method(':expand', '[]', 'path => true'))

```

```

# 全折叠
ui.button('全折叠', on_click=lambda: editor.run_editor_method('collapse', []))
# 切换只读模式
ui.button('设为只读', on_click=lambda: editor.run_editor_method('updateProps',
{'readOnly': True}))
ui.button('取消只读', on_click=lambda: editor.run_editor_method('updateProps',
{'readOnly': False}))

ui.run()

```

4. 事件监听

(1) 选择事件 (on_select)

选中 JSON 节点时触发，回调参数包含选中节点的路径、值等信息：

```

from nicegui import ui
from nicegui.events import JsonEditorSelectEventArguments

def on_node_select(e: JsonEditorSelectEventArguments):
    # e.path: 选中节点的路径（如 ["Address", "City"]）
    # e.value: 选中节点的值（如 "Wonderland"）
    ui.notify(f'选中节点路径: {e.path}, 值: {e.value}')

initial_json = {'Address': {'Street': 'Main Street', 'City': 'Wonderland'}}
editor = ui.json_editor(
    properties={'content': {'json': initial_json}},
    on_select=on_node_select
).classes('w-full h-64')

ui.run()

```

(2) 变更事件 (on_change)

JSON 数据修改时触发，回调参数包含更新后的数据：

```

from nicegui import ui
from nicegui.events import JsonEditorChangeEventArguments

def on_data_change(e: JsonEditorChangeEventArguments):
    ui.notify(f'数据变更: {e.value}')

editor = ui.json_editor(
    properties={'content': {'json': {'name': 'Alice', 'age': 42}}},
    on_change=on_data_change
).classes('w-full h-64')

ui.run()

```

四、进阶用法

1. 自定义编辑模式

JSONEditor 支持多种编辑模式，通过 `properties['mode']` 配置，常用模式：

- `'tree'`：树形结构（默认，可视化操作）。
- `'code'`：纯文本模式（手动编写 JSON 代码，支持语法高亮）。
- `'form'`：表单模式（适合非技术人员，通过表单输入修改数据）。

示例：切换为表单模式

```
from nicegui import ui

initial_json = {'name': 'Alice', 'age': 42, 'address': {'city': 'Wonderland'}}

editor = ui.json_editor(
    properties={
        'content': {'json': initial_json},
        'mode': 'form', # 表单模式
        'formOptions': {'inputs': {'string': {'type': 'text'}}} # 表单输入配置
    }
).classes('w-full h-96')

ui.run()
```

2. 数据导出与格式化

结合 `ui.download` 组件，将编辑后的 JSON 数据导出为文件，示例：

```
from nicegui import ui
import json

initial_json = {'name': 'Alice', 'age': 42, 'address': {'city': 'Wonderland'}}
editor = ui.json_editor(properties={'content': {'json': initial_json}}).classes('w-full h-64')

# 导出 JSON 文件
async def export_json():
    data = await editor.run_editor_method('get')
    # 格式化 JSON 字符串（缩进 2 空格）
    formatted_json = json.dumps(data, ensure_ascii=False, indent=2)
    ui.download(formatted_json, 'exported_data.json', 'application/json')

ui.button('导出 JSON 文件', on_click=export_json).classes('mt-4')

ui.run()
```

3. 限制额外字段 (Schema 配置)

通过 `additionalProperties: False` 禁止用户添加 Schema 中未定义的字段，确保数据结构严格符合要求：

```
from nicegui import ui

schema = {
    'type': 'object',
    'properties': {'id': {'type': 'integer'}, 'name': {'type': 'string'}},
    'required': ['id', 'name'],
    'additionalProperties': False # 禁止额外字段
}

editor = ui.json_editor(
    properties={'content': {'json': {'id': 1, 'name': 'Apple'}}},
    schema=schema
).classes('w-full h-64')

ui.run()
```

- 用户尝试添加 `'price'` 等额外字段时，编辑器会提示“不允许额外属性”。

4. 数组类型数据编辑

支持数组的增删改操作（添加元素、删除元素、修改元素值），结合 Schema 可约束数组元素类型，示例：

```
from nicegui import ui

# Schema 约束：数组元素必须为整数，且数组长度至少 2
schema = {
    'type': 'object',
    'properties': {
        'tags': {
            'type': 'array',
            'items': {'type': 'integer'}, # 数组元素必须为整数
            'minItems': 2 # 数组至少 2 个元素
        }
    }
}

initial_json = {'tags': [10, 20]}
editor = ui.json_editor(
    properties={'content': {'json': initial_json}},
    schema=schema
).classes('w-full h-64')

ui.run()
```

- 用户可通过编辑器的“+”按钮添加数组元素，若添加字符串（如 `'30'`），会提示类型错误。

五、核心方法速查表

方法	说明	参数	示例
run_editor_method(name, *args)	调用 JSONEditor 原生方法	name: 方法名 (前缀 ":" 表示参数为 JS 表达式) ; args: 方法参数	editor.run_editor_method('get') (获取数据)
update()	刷新编辑器显示	-	动态修改 properties 后调用, 确保界面同步
on_select(callback)	绑定节点选择回调	callback: 回调函数 (参数为 JsonEditorSelectEventArguments)	editor.on_select(lambda e: print(e.path))
on_change(callback)	绑定数据变更回调	callback: 回调函数 (参数为 JsonEditorChangeEventArguments)	editor.on_change(lambda e: print(e.value))
set_visibility(visible)	设置编辑器可见性	visible: 布尔值 (True/False)	editor.set_visibility(False) (隐藏编辑器)

常用 JSONEditor 原生方法 (通过 run_editor_method 调用)

方法名	说明	参数
get	异步获取当前 JSON 数据	-
expand	展开节点	第一个参数: 节点路径 ([] 表示所有节点) ; 第二个参数: 过滤函数
collapse	折叠节点	节点路径 ([] 表示所有节点)
updateProps	更新编辑器属性	配置字典 (如 {'readOnly': True})
set	设置 JSON 数据	新的 JSON 数据

六、注意事项与最佳实践

1. 数据类型兼容性

- JSON 不支持 Python 中的 tuple、datetime 等类型, 传递数据时需先转换为 JSON 支持类型 (如 tuple 转为 list, datetime 转为字符串)。
- 示例: 转换 datetime 类型

```
from datetime import datetime
from nicegui import ui

data = {
    'create_time': datetime.now().isoformat() # 转为 ISO 字符串
}

editor = ui.json_editor(properties={'content': {'json': data}}).classes('w-full h-64')
ui.run()
```


2. Schema 校验注意事项

- Schema 定义需严格遵循 JSON Schema 规范，否则可能导致校验失效。
- 复杂 Schema（如嵌套数组、条件约束）建议参考 [JSON Schema 官方文档](#) 编写。

3. 性能优化

- 大数据量 JSON（超过 1000 个节点）：建议使用 `'code'` 模式，关闭树形结构渲染，避免 DOM 元素过多导致卡顿。
- 频繁数据更新：减少 `on_change` 回调的复杂逻辑，可通过防抖优化（如 0.5 秒内仅触发一次）。

4. 版本兼容性

- `schema` 参数仅支持 NiceGUI 2.8.0 及以上版本。
- `html_id` 属性（用于自定义 CSS/JS）新增于 2.16.0 版本。
- 调用 JSONEditor 原生方法时，需确保方法名与 JSONEditor 版本兼容（NiceGUI 内置版本适配主流方法）。

总结

`ui.json_editor` 是 NiceGUI 中功能专业的 JSON 编辑组件，通过封装 JSONEditor 库实现了可视化编辑、Schema 校验、动态交互等核心能力。其核心优势在于：

- 无需手动处理 JSON 序列化 / 反序列化，直接操作 Python 字典，开发效率高；
- Schema 校验功能确保数据结构合规，适用于配置管理、接口参数编辑等场景；
- 支持调用原生方法扩展功能，灵活性强；
- 与 NiceGUI 生态无缝集成，可快速结合其他组件（如按钮、下载组件）实现完整流程。

适用于后台管理系统的配置编辑、接口调试工具的参数输入、JSON 数据可视化展示等场景。对于复杂需求（如自定义表单控件、多语言支持），可通过 JSONEditor 原生配置或插件进一步扩展。